

汇报 v3 — 平衡控制五大文件：协作机制与调用链深度剖析

第一部分：发言大纲

一句话概述

这个项目的平衡功能由 **5 个核心文件** 协同完成。它们通过 **硬件中断链 + 全局变量共享 + 函数调用** 三种机制紧密配合，实现每 5ms 一次的完整控制循环。

五个文件的分工

文件	角色	做什么
main.c	指挥官	初始化所有外设，然后进入死循环做显示和串口输出
exti.c	心跳起搏器	响应 MPU6050 的 5ms 中断，读传感器、数据融合、触发控制
control.c	大脑	执行 PD 直立环 + 速度阻尼算法，算出电机该给多少力
motor.c	肌肉	接收控制指令，做死区补偿，最终输出 PWM 驱动电机转动
gtim.c	脉搏计数器	5ms 定时读取编码器速度，提供速度环 PI 函数

第二部分：全局视角 — 从硬件中断到电机转动的完整链路

2.1 两条并行的 200Hz 时间线

系统中有**两个独立的 5ms 节拍**，它们来自不同的硬件，彼此不同步：

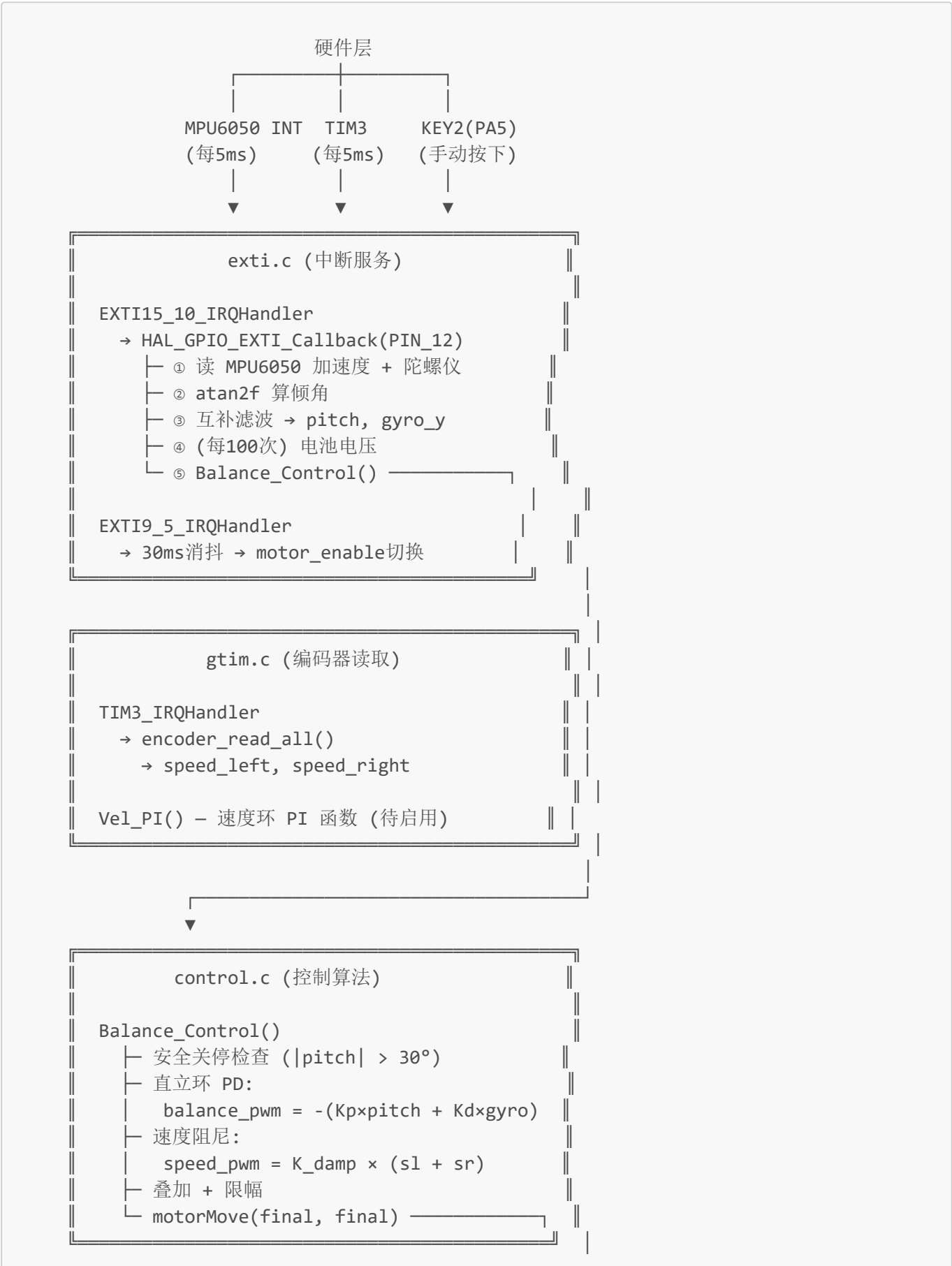
时间线 A（传感器 → 控制 → 电机）：
MPU6050 内部时钟
→ INT 引脚（PA12）拉低（每 5ms）
→ EXTI15_10 中断
→ exti.c：读传感器 → 互补滤波 → control.c：PID → motor.c：PWM

时间线 B（编码器读取）：
STM32 TIM3 计数器溢出（每 5ms）
→ TIM3 中断
→ gtim.c：读编码器 → speed_left, speed_right

为什么是两条独立的时间线而不是串在一起？

因为 MPU6050 INT 信号和 STM32 内部定时器是两个独立的时钟源。MPU6050 INT 确保"传感器数据是新鲜的"，TIM3 确保"编码器读取间隔精确"。在实际运行中，TIM3 的优先级更高（优先级 1 vs 2），所以 TIM3 可以打断 EXTI 的处理，确保编码器数据永远是最新的。

2.2 五个文件的调用关系全景图





2.3 全局变量：文件之间的"共享白板"

五个文件之间**不通过函数参数传递数据**，而是使用**全局变量**。这类似于一块所有人都能读写的白板：

全局变量分布图：

生产方	变量名	消费方
exti.c (ISR)	→ pitch (float)	→ control.c, main.c
exti.c (ISR)	→ gyro_y (float)	→ control.c, main.c
exti.c (ISR)	→ battery_voltage (float)	→ main.c
exti.c (ISR)	→ balance_enable (u8)	→ control.c, main.c
exti.c (ISR)	→ motor_enable (u8)	→ motor.c, control.c, main.c
gtim.c (ISR)	→ speed_left (int)	→ control.c, main.c
gtim.c (ISR)	→ speed_right (int)	→ control.c, main.c
control.c	→ debug_balance_pwm (int)	→ main.c
control.c	→ debug_speed_pwm (int)	→ main.c
control.c	→ debug_final_left (int)	→ main.c
control.c	→ debug_final_right (int)	→ main.c

```
motor.c      → motor_left (int)      → main.c
motor.c      → motor_right (int)     → main.c
```

为什么用全局变量而不是函数参数？

因为生产方是**中断服务函数**（ISR）。中断函数由硬件触发，没有“调用者”可以传参。中断修改全局变量，其他代码读取——这是嵌入式系统中最常见的 ISR 通信方式。

这里有一个潜在的问题：TIM3 中断（优先级 1）可以在 EXTI 中断（优先级 2）执行 `Balance_Control()` 的过程中抢夺 CPU，此时 `speed_left` / `speed_right` 会被更新。但由于每个变量是单次读写（int 类型，一次指令完成），且速度数据仅用于阻尼（不是精确控制），所以不会造成实际问题。

第三部分：逐文件详细剖析

3.1 exti.c — 系统的“心跳起搏器”

它在整个系统中的位置

`exti.c` 是控制循环的**唯一入口**。没有它，后面的 `control`、`motor` 都不会执行。它是从“硬件世界”进入“软件世界”的第一道门。

初始化调用链

```
main.c: main()
├─ atk_ms6050_init()   ← 先初始化 MPU6050（配置 200Hz、使能 INT）
├─ exti_init()         ← 配置 PA12 为下降沿中断
└─ extix_key2_init()   ← 配置 PA5 为下降沿中断（按键）
```

`exti_init()` 和 `extix_key2_init()` 内部做三件事：

1. 配置 GPIO 引脚模式（PA12 和 PA5 都设为中断输入 + 上拉）
2. 设置 NVIC 优先级（都为 2）
3. 使能对应的中断线（EXTI15_10 和 EXTI9_5）

为什么 `exti_init` 必须在 `MPU6050_init` 之后调用？ MPU6050 的 INT 引脚在初始化完成前可能处于不确定状态，如果先使能中断可能触发伪中断。先初始化传感器，再打开中断，确保第一个中断就是有效的数据就绪信号。

中断响应：从中断触发到函数调用的完整路径

当 MPU6050 完成一次采样，INT 引脚从高变低。STM32 的 EXTI 硬件检测到这个下降沿：

```
硬件：PA12 引脚电平 ↓
|
| （硬件自动：EXTI 边沿检测 → 挂起寄存器置位 → NVIC 仲裁）
|
```

```

▼
中断向量表 → EXTI15_10_IRQHandler()    ← exti.c:115
|
|  HAL  框架层
▼
HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_12)  ← HAL  库函数
|
|  清除中断挂起位
|  根据 Pin 号分发
▼
HAL_GPIO_EXTI_Callback(GPIO_PIN_12)    ← exti.c:58, 我们的代码

```

关键点：中断向量表里的 `EXTI15_10_IRQHandler` 是硬件规定的固定函数名。你在 `startup_stm32f103xb.s` 中找到这个名字。HAL 库提供了默认的弱定义 (`__weak`)，我们在 `exti.c` 中覆盖了它。

`HAL_GPIO_EXTI_IRQHandler` 是 HAL 的中断分发函数，它会清除硬件中断标志，然后调用我们写的 `HAL_GPIO_EXTI_Callback`。

Callback 内部的完整执行流程（每 5ms 执行一次）

```

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_12) {
        // =====
        // 第 1 步：读取传感器原始数据（通过 I2C）
        // =====
        int16_t ax, ay, az, gx, gy, gz;
        atk_ms6050_get_accelerometer(&ax, &ay, &az); // 读 6 字节加速度
        atk_ms6050_get_gyroscope(&gx, &gy, &gz);      // 读 6 字节陀螺仪

        // =====
        // 第 2 步：计算加速度计倾角
        // =====
        float accel_angle = atan2f((float)ay, (float)az) * 57.29578f;
        // atan2f: 返回弧度, ×57.3 = 转换为度数
        // 用 AY 和 AZ 两个轴: AY 指向前方, AZ 指向下方
        // 车身水平时 AY≈0, AZ≈-1g → atan2(0, -1) = 0°
        // 前倾时 AY 变大 → 角度为正

        // =====
        // 第 3 步：陀螺仪原始值 → 物理单位转换
        // =====
        gyro_y = (float)gx / 16.4f;
        // MPU6050 陀螺仪量程设为 ±2000°/s
        // 灵敏度 = 2000 / 32768 = 0.061 (°/s)/LSB
        // 或者等价: 16.4 LSB/(°/s)
        // 即: 原始值 ÷ 16.4 = 每秒度数

        // =====
        // 第 4 步：互补滤波 — 核心算法
        // =====
        static float comp_angle = 0; // static! 保持上一次的值
    }
}

```

```

    comp_angle = 0.95f * (comp_angle + gyro_y * 0.005f)
                + 0.05f * accel_angle;
    pitch = comp_angle;

    // 拆解:
    // (comp_angle + gyro_y * 0.005)
    //   = 上次角度 + 角速度 * 5ms
    //   = 陀螺仪积分 → 对短期变化响应快
    // 0.95 * 陀螺积分 + 0.05 * 加速度计
    //   = 95% 信任陀螺仪 (平滑), 5% 信任加速度计 (消除漂移)

    // =====
    // 第 5 步: 电池电压 (降频到每 500ms 一次)
    // =====
    static uint8_t volt_cnt = 0;
    volt_cnt++;
    if (volt_cnt >= 100) {
        volt_cnt = 0;
        battery_voltage = readVolt(10); // 读 10 次取平均
        balance_enable = 1; // 暂时禁用低压保护
    }

    // =====
    // 第 6 步: 调用控制算法 (跨文件调用!)
    // =====
    extern void Balance_Control(void); // 声明外部函数
    Balance_Control(); // → 跳转到 control.c
}

if (GPIO_Pin == GPIO_PIN_5) {
    // KEY2 按键处理 (30ms 消抖)
    // ...
    motor_enable = !motor_enable; // 切换电机状态
}
}

```

关键设计决策

为什么把互补滤波放在 exti.c 而不是 MPU6050 驱动里?

MPU6050 驱动 (`atk_ms6050.c`) 是一个“纯”传感器驱动——它的职责只是和硬件通信，读取原始数据。姿态解算 (互补滤波) 是应用层逻辑，不同的项目可能需要不同的滤波算法 (互补滤波、卡尔曼滤波、Madgwick 等)。把滤波放在 `exti.c` 中，换滤波算法时不需要改动 MPU6050 驱动。

`comp_angle` 为什么是 `static`?

`static` 局部变量的生命周期是整个程序运行期，不会在函数返回后被销毁。互补滤波需要上一次融合后的角度值来做陀螺仪积分。如果不用 `static`，每次进入 Callback 时 `comp_angle` 都会被重置为 0。

为什么电池电压 100 次才读一次?

I2C 通信耗时较长。如果每次 5ms 中断都读电池（还需要通过 ADC 转换），会增加中断执行时间，可能影响控制周期。500ms 更新一次电压对显示来说完全够用。

3.2 control.c — 系统的"大脑"

它在整个系统中的位置

`control.c` 的 `Balance_Control()` 函数被 `exti.c` 的中断回调调用。它读取 `exti.c` 提供的 `pitch` 和 `gyro_y`，读取 `gtim.c` 提供的 `speed_left` 和 `speed_right`，经过计算后调用 `motor.c` 的 `motorMove()`。

它不接触任何硬件。不读传感器，不写寄存器。它只做纯数学计算。这也是分层架构的好处——控制算法可以脱离硬件独立测试。

Balance_Control() 逐层分析

```
void Balance_Control(void)
{
    int balance_pwm;    // 直立环输出
    int speed_pwm;      // 速度阻尼输出
    int speed_sum;       // 左右轮速度之和
    int final_left, final_right; // 最终左右 PWM

    // =====
    // 第一层：安全检查
    // =====

    // 检查 1：系统是否允许平衡控制？
    if (!balance_enable) { // balance_enable 由 exti.c 设置
        turn_Off();        // 关闭所有电机输出
        return;             // 直接返回，不做任何计算
    }

    // 检查 2：车身是否倒了？（倾角超过 30°）
    if (pitch > SAFE_ANGLE_MAX || pitch < -SAFE_ANGLE_MAX) {
        turn_Off();        // 倒了就别转了，防止轮子空转损坏电机
        return;
    }

    // =====
    // 第二层：直立环 PD 计算
    // =====
    balance_pwm = -(int)(Balance_Kp * (pitch - MECHANICAL_ZERO)
                        + Balance_Kd * gyro_y);

    // 拆解这个公式：
    //
    // P 项：Balance_Kp × (pitch - MECHANICAL_ZERO)
    // pitch = 当前倾角（前倾为正）
    // MECHANICAL_ZERO = 物理平衡点角度（-0.10°）
    // 如果车前倾了 2°：P项 = 300 × (2.0 - (-0.1)) = 300 × 2.1 = 630
    // 效果：倾角越大，回正的力越大
```

```

//
// D 项: Balance_Kd × gyro_y
//   gyro_y = 当前角速度 (前倾时为正)
//   如果车正在以 50°/s 前倾: D项 = 7.0 × 50 = 350
//   效果: 倒得越快, 刹车力越大
//
// 负号 (-): 把"需要回正的方向"转换成电机转动的方向
//   车身前倾 → 需要轮子向前转 → 根据电机接线定义正转方向
//
// P 和 D 叠加的效果:
//   前倾 2° + 正以 50°/s 倒下 → 630 + 350 = 980 的 PWM 去回正
//   前倾 2° + 正以 -10°/s 回正 → 630 + (-70) = 560 (D 项在"刹车")

// =====
// 第三层: 速度阻尼计算
// =====
speed_sum = speed_left + speed_right;
// speed_left/right 由 gtim.c 的 TIM3 中断每 5ms 更新
// speed_sum > 0 表示车整体在往前移动

// 死区: 小速度不响应 (避免传感器噪声导致不必要的干预)
if (speed_sum > -SPEED_DAMP_DEADZONE && speed_sum < SPEED_DAMP_DEADZONE) {
    speed_sum = 0; // |speed| < 3 → 忽略
}

// 纯比例阻尼 (注意: 没有积分项!)
speed_pwm = (int)(SPEED_DAMP_K * (float)speed_sum);
// SPEED_DAMP_K = 6.0
// 如果车在往前移动 (speed_sum > 0), speed_pwm 为正
// speed_pwm 会被加到 final 中, 起到"拉回来"的效果

// =====
// 第四层: 合并输出
// =====
PWM_Limit(&balance_pwm); // 限幅到 ±7199

final_left = balance_pwm + speed_pwm;
final_right = balance_pwm + speed_pwm;
// 两轮指令完全相同 (平衡车不需要差速转弯)
// 合并逻辑:
//   直立环说"往前转 500" + 速度阻尼说"往后拉 100"
//   → 最终往前转 400

PWM_Limit(&final_left);
PWM_Limit(&final_right);

// DEBUG_PWM_LIMIT: 调参保护限幅 (当前设为 7199 = 不限制)
if (final_left > DEBUG_PWM_LIMIT) final_left = DEBUG_PWM_LIMIT;
if (final_left < -DEBUG_PWM_LIMIT) final_left = -DEBUG_PWM_LIMIT;
// ... 同理对 final_right

// =====
// 第五层: 存储调试变量
// =====

```



```

debug_balance_pwm = balance_pwm;

// 这些全局变量只用于调试输出，不影响控制逻辑
// main.c 的 50ms 循环会读取它们并输出到串口

// =====
// 第六层：驱动电机
// =====
if (motor_enable && !DEBUG_SENSOR_ONLY) {
    motorMove(final_left, final_right); // → 跳转到 motor.c
} else {
    turn_Off(); // 电机关闭或调试模式
}
}

```

关键设计决策

为什么不在这里调用 `Vel_PI()` 而是自己写了速度阻尼？

`Vel_PI()`（在 `gtim.c` 中）是正式的增量式 PI 控制器，包含积分项。但在直立环没调好的阶段，积分项会“记仇”——它在车被推离原位时累积误差，然后在车试图回正时反向输出，导致车反而倒向另一边。当前的速度阻尼只用纯比例（P），速度快了就拉一下，不累积历史误差。

这也是为什么 `Velocity_Kp` 和 `Velocity_Ki` 虽然定义为全局变量，但当前值都是 0 的原因。

`PWM_Limit` 和 `DEBUG_PWM_LIMIT` 双重限幅的意义？

`PWM_Limit` 是硬件级别的保护（ ± 7199 = PWM 寄存器最大值，超过会溢出）。`DEBUG_PWM_LIMIT` 是调参级别的保护——比如可以临时设为 500，防止调参初期参数不当导致车猛冲。当前 `DEBUG_PWM_LIMIT` 已调到 7199（等于无限制）。

3.3 motor.c — 系统的“肌肉”

它在整个系统中的位置

`motor.c` 是控制链的最末端。它接收 `control.c` 传过来的 PWM 目标值（`final_left`, `final_right`），把它变成实际的电信号驱动电机。

motorMove() 逐层分析

```

void motorMove(int speedL, int speedR)
{
    // =====
    // 第一层：保存原始值（用于调试输出）
    // =====
    motor_left = speedL;
    motor_right = speedR;
    // 这两个全局变量后来被多次覆盖（限幅、增益、死区），
    // 但保存在这里可以让 main.c 看到“进入 motor 前的原始值”
}

```

```

// =====
// 第二层：硬件级限幅
// =====
PWM_Limit(&motor_left);    // 限制到 [-7199, +7199]
PWM_Limit(&motor_right);

// =====
// 第三层：左右增益校准
// =====
motor_left  = (int)(motor_left * MOTOR_LEFT_GAIN);    // ×1.00
motor_right = (int)(motor_right * MOTOR_RIGHT_GAIN);  // ×1.00
// 两个电机物理特性不同，相同 PWM 下转速可能不一样
// 如果一个电机偏慢，把它的 GAIN 调大（如 1.05）

// =====
// 第四层：死区补偿（核心难点）
// =====
motor_deadzone(&motor_left,  MOTOR_LEFT_DZ);    // DZ=100
motor_deadzone(&motor_right, MOTOR_RIGHT_DZ);    // DZ=100

// motor_deadzone 内部逻辑：
//   if (pwm > 5)      pwm += 100;  // 正向，叠加死区补偿
//   else if (pwm < -5) pwm -= 100;  // 反向，叠加死区补偿
//   else              pwm = 0;      // 信号太小（|pwm|≤5），不转
//
// 例：control 输出 +50（小信号，电机可能不转）
//     → 50 > 5 → 50 + 100 = 150 → 确保电机能转起来
//
// 例：control 输出 +3（传感器噪声）
//     → 3 ≤ 5 → 0 → 忽略，电机不动

// =====
// 第五层：安全限幅
// =====
if (motor_left > MOTOR_SAFE_LIMIT)  motor_left = MOTOR_SAFE_LIMIT;
if (motor_left < -MOTOR_SAFE_LIMIT) motor_left = -MOTOR_SAFE_LIMIT;
// MOTOR_SAFE_LIMIT 当前 = 7199（等于硬件最大值）

// =====
// 第六层：写入硬件（方向引脚 + PWM 占空比）
// =====

// 左电机：
if (motor_left >= 0) {
    AIN1(1); AIN2(0);    // 方向：正转
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, motor_left);
    // 把 motor_left 写入 TIM1 的 CCR1 寄存器
    // PA8 输出占空比 = motor_left / 7200 × 100%
} else {
    AIN1(0); AIN2(1);    // 方向：反转
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, -motor_left);
    // 取绝对值写入 CCR（占空比不能为负）
}

```

```
// 右电机：同理，用 TIM_CHANNEL_4 (PA11)
}
```

死区补偿的演化史（调试过程中尝试过的三种方案）

方案 1：硬阈值（已废弃）

```
if (pwm > 0 && pwm < 285)  pwm = 285;
```

- 问题：小角度倾角时 control 输出 50~200，全部被拉到 285。导致 1° 倾角就输出很大力，车来回猛冲。
- 失败原因：小角度时"要么不转，要么过猛"，没有中间地带。

方案 2：偏移式（已废弃）

```
if (pwm > 0)  pwm += 285;
```

- 问题：虽然小角度有响应了，但微小的抖动（如 pwm=10）也被放大到 295，"手都按不住"。
- 失败原因：传感器噪声被放大。

方案 3：柔和死区（当前使用）

```
if (pwm > 5)      pwm += 100;
else if (pwm < -5)  pwm -= 100;
else              pwm = 0;
```

- 参数说明：
 - `MOTOR_DZ_START = 5`：信号太小 ($|pwm| \leq 5$) 就不驱动，靠自然摩擦保持静止
 - `MOTOR_DZ = 100`：一旦决定要转，额外加 100 确保跳过死区
- 为什么是 100 而不是 285？
 - 实际死区是 285 左右，但你不需要全补偿
 - 因为 PID 的 P 项本身也在增大输出，叠加后足够克服静摩擦
 - 补偿 100 刚好弥补"电机从不动到能动的最大缺口"

关键设计决策

为什么方向控制用 GPIO 而不是 PWM 的互补输出？

STM32 高级定时器（TIM1）有硬件互补输出（CH1 + CH1N），可以自动生成带死区的两路 PWM，适合驱动半桥。但 TB6612 芯片内部已经有 H 桥，只需要方向 + PWM 两路信号。手动控制方向引脚更灵活，而且不浪费一个互补输出通道。

3.4 gtim.c — 系统的"脉搏计数器"

它在整个系统中的位置

gtim.c 有两个独立的功能：

1. **编码器数据采集**（通过 TIM3 5ms 中断自动执行）— 向 control.c 提供速度数据
2. **速度环 PI 函数 Vel_PI()**（暂未启用）— 提供速度闭环控制

功能一：TIM3 中断 → 编码器读取

```
// 初始化时配置的中断周期：
// Prescaler = SystemCoreClock / 10000 - 1    （自适应时钟）
// Period = 49
// → 中断频率 = 10KHz / 50 = 200Hz = 每 5ms 一次

// 中断触发链：
// 硬件：TIM3 计数器从 0 计到 49
//   |
//   ▼
// TIM3_IRQHandler()                ← gtim.c:66
//   |
//   ▼
// HAL_TIM_IRQHandler(&htim3)       ← HAL 库
//   |    （检查中断标志，确认是"周期溢出"事件）
//   ▼
// HAL_TIM_PeriodElapsedCallback(htim) ← gtim.c:71
//   |
//   ▼
//   encoder_read_all();              ← encoder.c:86
//   |
//   └─ speed_left  = Read_Encoder(&htim2);
//   └─ speed_right = Read_Encoder(&htim4_encoder);
```

为什么用定时器中断而不是在 EXTI 中断中读编码器？

两个原因：

1. **优先级更高**（TIM3=1 > EXTI=2），确保编码器数据永远是最新的。即使 EXTI 中断正在做 PID 计算，TIM3 也能打断它更新速度值。
2. **解耦**。编码器读取是独立功能，不应该和姿态传感器耦合。如果以后换用 SPI 接口的 IMU 导致 EXTI 中断变长，编码器读取频率不受影响。

功能二：Vel_PI() 增量式 PI 控制器

```
int Vel_PI(int vel, int Target)
{
    // vel = 当前速度（编码器读数），Target = 目标速度（通常是 0）

    int error = Target - vel;
    static int last_error = 0;    // 上一次的误差
    static int pwm_out = 0;      // 累计输出

    // 安全检查：电机关闭时清零
    if (!motor_enable) {
```

```

        last_error = 0;
        pwm_out = 0;
        return 0;
    }

    // 增量式 PI 公式
    int delta = (int)(Velocity_Kp * (error - last_error) + Velocity_Ki * error);
    //          ↑                ↑
    //          P 项: 误差的变化量    I 项: 当前误差

    pwm_out += delta; // 在上次输出基础上叠加增量

    // 抗积分饱和
    if (pwm_out > PWM_MAX) pwm_out = PWM_MAX;
    if (pwm_out < PWM_MIN) pwm_out = PWM_MIN;

    last_error = error;
    return pwm_out;
}

```

为什么增量式 PI 需要 `static` 变量?

`last_error` 和 `pwm_out` 需要在多次调用之间保持。`last_error` 用于计算"误差的变化量", `pwm_out` 是累计输出 (增量式 = 输出增量 + 上次输出)。如果用普通局部变量, 每次调用都被重置, PI 就退化成了纯 P。

它当前为什么没有被调用?

在 `control.c` 的 `Balance_Control()` 中, 速度处理用的是 `SPEED_DAMP_K × speed_sum` (纯比例阻尼), 没有调用 `Vel_PI()`。这是有意为之——`Vel_PI()` 是"正式速度环", 在当前直立环没调稳的阶段启用会导致前面分析过的"推偏平衡点"问题。

启用时机: **直立环调好后** (松手能站 3-5 秒不倒), 再在 `Balance_Control()` 中改为调用 `Vel_PI(speed_sum, target_speed)` 来替代 `SPEED_DAMP_K × speed_sum`。

3.5 main.c — 系统的"指挥官"

它在整个系统中的位置

`main.c` 的角色是**启动和监控**。它不参与 5ms 控制循环, 只做两件事:

1. **启动时**: 按正确顺序初始化所有外设
2. **运行时**: 每 50ms 从全局变量读取数据, 刷新 OLED 和串口

初始化顺序: 一个都不能乱

```

int main(void)
{
    // =====
    // 阶段 1: 系统基础设施 (必须最先)
    // =====
}

```

```

HAL_Init(); // ① HAL 库自己的初始化
sys_stm32_clock_init(9); // ② 配置时钟 → 64MHz
delay_init(SystemCoreClock/1000000); // ③ 延时初始化（依赖时钟频率）

// =====
// 阶段 2：人机界面（尽早可用，方便调试）
// =====
oled_init();
oled_show_string(0, 0, "Init...", 16);
oled_refresh_gram(); // 显示 "Init..." 表示系统开始启动

usart_init(115200);
printf("Balance Car Starting...\r\n"); // 串口输出启动信息

// =====
// 阶段 3：外设初始化（顺序灵活）
// =====
led_init(); // LED
motor_init(); // 电机 PWM (TIM1)
leftEncoderInit(); // 左编码器 (TIM2)
rightEncoderInit(); // 右编码器 (TIM4)
gtim_init(); // 5ms 定时器 (TIM3) → 开始产生中断
adc_init(); // 电池 ADC

// =====
// 阶段 4：传感器 + 中断（最后初始化）
// =====
atk_ms6050_init(); // MPU6050（配置 200Hz，使能 INT 引脚）
printf("MPU6050 OK.\r\n");

exti_init(); // PA12 EXTI → 开始响应 MPU6050 中断！
extix_key2_init(); // PA5 EXTI → 开始响应按键中断！
printf("EXTI OK.\r\n");

// 到这里，系统已完全启动。5ms 控制循环自动在后台运行。
// 下面进入前台显示循环。

// =====
// 阶段 5：主循环（永远执行）
// =====
printf("DBG,t,p100,g10,s1,sr,ss,bp,sp,fl,fr,ml,mr,en,be\r\n");
// 输出 CSV 表头

uint32_t last_show = 0;

while (1)
{
    if (HAL_GetTick() - last_show >= 50) { // 每 50ms 执行一次
        last_show = HAL_GetTick();

        // — OLED 显示 —
        // 第 1 行：倾角
        oled_show_string(0, 0, "Pitch:", 16);
        sprintf(dis_buf, "%.2f", pitch); // pitch 来自 exti.c 全局变量
    }
}

```

```
oled_show_string(48, 0, disp_buf, 16);

// 第 2 行: 速度
oled_show_string(0, 16, "Speed:", 16);
oled_show_num(48, 16, speed_left + speed_right, 4, 16);
// speed_left/right 来自 gtim.c 全局变量

// 第 3 行: 电池电压
// 第 4 行: 电机状态

oled_refresh_gram(); // 关键: 推送到屏幕!

// — 串口 CSV 输出 —
printf("D,%lu,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d\r\n",
    HAL_GetTick(),
    (int)(pitch * 100.0f), // 倾角 ×100
    (int)(gyro_y * 10.0f), // 角速度 ×10
    speed_left, // 左编码器
    speed_right, // 右编码器
    speed_left + speed_right, // 速度之和
    debug_balance_pwm, // 直立环输出
    debug_speed_pwm, // 速度阻尼输出
    debug_final_left, // 左目标 PWM
    debug_final_right, // 右目标 PWM
    motor_left, // 左实际 PWM
    motor_right, // 右实际 PWM
    motor_enable, // 电机开关
    balance_enable); // 平衡使能
    }
}
}
```

初始化顺序中的依赖关系

如果顺序错了会出问题:

错误顺序	后果
oled_init 在 sys_stm32_clock_init 之前	OLED 驱动用的延时函数 (delay_us) 依赖系统时钟, 此时时钟还没配置
exti_init 在 atk_ms6050_init 之前	MPU6050 还没配置好, INT 引脚状态不确定, 可能收到伪中断
gtim_init 在 encoder_init 之前	gtim 的 TIM3 中断里调用 encoder_read_all(), 但编码器定时器没初始化 → 读垃圾数据
printf 在 usart_init 之前	串口还没配置, printf 会卡死或输出到未知设备

主循环中的"非阻塞定时"

```
if (HAL_GetTick() - last_show >= 50) {
    last_show = HAL_GetTick();
    // ... 执行任务
}
```

这不是 `delay_ms(50)`。`delay_ms` 会让 CPU 空转 50ms，期间不能做任何其他事。而非阻塞方式让 CPU 在两次显示之间可以处理中断——实际上，在这 50ms 内，**10 次完整的 5ms 控制循环已经执行完毕**。

换句话说，你看到的每一行串口数据，是过去 50ms 内 10 次控制循环的**最后一拍快照**。

第四部分：完整控制时序 — 一个 10ms 切片内的真实故事

假设小车正被扶着，某时刻松手。下面是在松手后最初 10ms（两个控制周期）内，五个文件之间发生的一切：

```
t = 0ms    松手！小车开始前倾。

t = 0~5ms  第 1 个控制周期：

[MPU6050]  加速度计检测到 AY 分量开始增大
            INT 引脚保持高电平（数据还没准备好）

[TIM3]      计数器在跑，还没到 49

[main.c]    主循环在执行第 N 次 OLED 刷新

t = 5.0ms  第 1 个控制周期触发：

[MPU6050]   数据采样完成！INT 引脚 ↓

[stm32]     EXTI 硬件检测到 PA12 下降沿
            → NVIC 判断优先级 → 保存 main 现场
            → 跳转到 EXTI15_10_IRQHandler

[exti.c]    HAL_GPIO_EXTI_Callback(PIN_12):
            ① 读加速度计：ay=500, az=-16000（前倾了约 1.8°）
            ② 读陀螺仪：    gx=2000（正以约 122°/s 前倾！）
            ③ atan2f(500, -16000) × 57.3 = -1.79°
               注意符号：ay 为正表示前倾
            ④ gyro_y = 2000 / 16.4 = 122.0 °/s
            ⑤ 互补滤波：
               上次 comp_angle = 0.0（扶着时是水平的）
               comp_angle = 0.95×(0.0 + 122.0×0.005) + 0.05×(-1.79)
                           = 0.95×0.61 + 0.05×(-1.79)
                           = 0.58 + (-0.09)
                           = 0.49°
               → pitch = 0.49°
```


volt_cnt=0, 不读电池电压。

⑥ 调用 Balance_Control()

```
[control.c] Balance_Control():
    pitch=0.49°, gyro_y=122.0°/s
    speed_left=0, speed_right=0
    balance_enable=1, |pitch|<30° ✓

    P项 = 300 × (0.49-(-0.10)) = 300×0.59 = 177
    D项 = 7.0 × 122.0 = 854
    balance_pwm = -(177+854) = -1031
    (负号: 车在快速前倾, 需要一个很大的反向力)

    speed_sum = 0+0 = 0 → speed_pwm = 0
    final = -1031 + 0 = -1031 (限幅后不变)

    motorMove(-1031, -1031)

[motor.c] motorMove():
    motor_left = -1031
    PWM_Limit: -1031 在 [-7199, 7199] ✓
    增益 ×1.0 → 不变
    motor_deadzone: -1031 < -5 → -1031-100=-1131
    SAFE_LIMIT: -1131 在范围内 ✓
    motor_left < 0 → AIN1=0, AIN2=1
    TIM1_CCR1 = 1131 → PA8 输出 PWM
    (占空比 1131/7200 = 15.7%)
    右电机同理

[电机] 开始反转, 产生一个向后的力
        把前倾的车身往回"推"

[stm32] 中断返回 → 恢复 main 现场
```

t = 5.0~5.1ms 几乎同时发生:

```
[TIM3] 计数器到达 49 → 溢出!
        → NVIC 优先级 1 > EXTI 的优先级 2
        等等, EXTI 中断刚结束, 两者没冲突。
        → 进入 TIM3_IRQHandler
```

```
[gtim.c] HAL_TIM_PeriodElapsedCallback:
        → encoder_read_all()
        → speed_left = Read_Encoder(&htim2)
        读 TIM2 计数器 (已清零) → 0 (车还没明显移动)
        → speed_right = 0
```

t = 5~10ms 第 1 控制周期结束。车正在被电机推回。
 motorMove 设置的 PWM 持续输出。
 车身倾角开始减小。
 main.c 继续跑 OLED 刷新 (50ms 还没到, 不执行显示)。

t = 10.0ms 第 2 个控制周期触发:

[MPU6050] INT 再次 ↓

[exti.c] 加速度计: $ay=300$, $az=-16200$ (倾角变成了 1.06°)
陀螺仪: $gx=500$ (角速度降到 $30.5^\circ/s$, 正在回正!)
互补滤波: $pitch = 0.95 \times (0.49 + 30.5 \times 0.005) + 0.05 \times (-1.06) = 0.58^\circ$
(角度还在增大, 因为车的惯性)

[control.c] $P\text{项} = 300 \times 0.68 = 204$
 $D\text{项} = 7.0 \times 30.5 = 213$ ← D 项大幅减小, 因为不快速倒下了
 $balance_pwm = -(204 + 213) = -417$
输出比上一个周期小了很多 ($-1031 \rightarrow -417$)

[motor.c] `motorMove(-417, -417)`
→ `motor_deadzone: -417-100 = -517`
→ `PWM = 7.2%`

→ 车的回正速度在减慢, 避免"冲过头"

`t = 50ms` `main.c` 刷新 OLED 和串口:
输出一行 `D,...` 数据
此时你可以看到: `p100≈58`, `g10≈305`, `m1=mr≈-517`

从这个时序可以看出:

1. **D 项是第一时间响应最快的:** 松手瞬间 `gyro_y` 很大 ($122^\circ/s$), D 项立刻输出 854 的刹车力。而 P 项只有 177。如果只有 P 没有 D, 车要等到倾角很大才知道回正, 那时候已经来不及了。
2. **一个控制周期内的决策链极快:** 从 MPU6050 INT 触发到 PWM 更新, 全部在中断中完成, 延迟 $< 0.1ms$ (约 6000 个 CPU 周期)。
3. **速度阻尼还没发挥作用:** 前两个周期 `speed_sum` 都是 0 (车刚开始移动), 要等到编码器检测到明显位移才有输出。

第五部分：发言要点（按此结构讲）

开场（30秒）

"这个项目的平衡功能由 5 个文件协同完成, 它们通过中断链 + 全局变量 + 函数调用三种方式协作, 实现每 5ms 一次的控制循环。"

核心内容（3-4 分钟）

第一张图：文件协作关系

边画边讲, 用箭头标出调用方向和变量流动。

第二张图：一个 5ms 控制周期的执行流

MPU6050 INT 触发 → exti 读传感器 → 互补滤波
→ control PD 计算 → motor 死区补偿 → TIM1 PWM → 电机转动

每一步用一句话说清楚做了什么，为什么在那个文件里做。

重点技术点（挑 2-3 个讲）

1. **互补滤波为什么放在 exti.c?** — 传感器驱动只管读数据，算法层管数据融合，分开了才能独立维护。
2. **死区补偿为什么经历了三种方案?** — 课本上没有的工程实践。理论告诉你需要补偿，但没告诉你补偿多少、用哪种方式。这是需要通过实验来确定的。
3. **为什么速度环要先关掉?** — 串级 PID 的"先内后外"原则。积分项在直立不稳时反而帮倒忙。这个教训是实际调出来的。

总结（30秒）

"五个文件分工明确——exti 是心跳、control 是大脑、motor 是肌肉、gtim 是脉搏、main 是眼睛。它们之间的接口是全局变量和函数调用。目前所有驱动正常，正在用串口 CSV 数据指导 PID 调参。"

基于 Template/ 下 exti.c、motor.c、gtim.c、control.c、main.c 五个核心文件的完整源码追踪和调用链分析。